

-Introduction –

In our exploration of computer architecture, we began by learning to implement the S12 Instruction Set Architecture (ISA), which comprises 12 simple instructions necessary for writing programs. These instructions include operations like ADD, AND, OR, JMP, and others that facilitate instruction-level programming. Utilizing this set of instructions helps us manage complexity as we work to understand computer architecture and its purpose.

For this project, we established two benchmarks to implement a pipeline approach to programming with the ISA. This method involves stacking instructions to reduce the time required to execute the instruction set. The process begins with fetching the first instruction, which is then sent for decoding. While the first instruction is being decoded, the second instruction is fetched. This assembly line setup continues until all instructions are completed. We aim to decrease the execution time and, consequently, the cycles per instruction (CPI).

The simplicity of the S12 ISA has a significant advantage; it makes implementing a pipeline easier compared to more complex ISAs. However, this limited set of instructions can also pose challenges, as some programs may require more instructions to complete a task that could be efficiently accomplished with a single, more complex instruction.

Throughout this project, we observed a range of results, with some outcomes being more favorable than others. Nevertheless, most findings aligned with our original hypotheses, allowing us to deepen our understanding of the subject.

Our pipelined processor design for the S12 ISA follows a five-stage structure: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), and Write-Back (WB). Each stage is separated by a latch structure (IF_ID, ID_EX, EX_MEM, MEM_WB) that stores both data (e.g., PC, ACC, operands, memory addresses) and control signals (MemRead, MemWrite, RegWrite, BranchJump, etc.). This allows independent progression of up to five

instructions through the pipeline simultaneously. The datapath supports all 12 S12 instructions, including indirect operations (LOADI, STOREI). Figure 1 (in the report) shows the datapath with pipeline registers and the forwarding paths from MEM/WB→EX that resolve most read-after-write (RAW) hazards without stalling.

Design choices were made to balance correctness, clarity, and simplicity. A five-stage model was chosen because it best exposes instruction overlap while keeping the control logic manageable. Forwarding is implemented from MEM/WB stages to the EX stage, reducing stalls for arithmetic and memory operations that immediately consume ACC results. Load-use hazards are detected in the ID stage and resolved by inserting a single stall cycle. Branches are resolved in EX to simplify control; a taken branch flushes IF and ID to avoid fetching along the wrong path. These choices minimize control complexity while maintaining visible pipelining benefits.

- BranchJmp - asserted for JMP, JN, and JZ; triggers PC redirection
- MemRead/MemWrite - flags for loads/stores
- RegWrite - is ACC used
- UseIndir - direct/indirect addressing
- ALU operations - describes the instruction

Each control signal propagates through the pipeline registers to ensure the correct operation at later stages, and the hazard-detection logic checks these signals to decide when to stall or forward

The software design of the pipelined simulator builds directly on the earlier instruction-level simulator, but with the internal state restructured around the five pipeline stages instead of a single fetch-decode-execute loop. Both simulators share the same instruction set but execute instructions differently. The new simulator passes instructions from one of five stages to

the next. There are latches between the stages that pass data and describe the state of the machine. The simulator also adds realism in that it has to stall when it asks for information from memory, so the machine must account for that. I used the reading for help with some of the design and had to make some compromises on the design like the tick order so I could pass my information in the correct order to catch the right data at the right time.

DATA

Benchmark multiply_space

Benchmark Characteristics

Instruction Mix

60% Mem, 10% ALU, 20% Conditional, 10% Other

Number Instructions

10

Performance

Number Cycles

51

Instruction mix

45% Mem, 22% ALU, 22% Conditional, 2% Other

Comparison

The CPI went up in the pipelined program, but this is due to stalls caused by hazards, and because in the previous ISA, we assumed a default CPI of 1. If this were a real comparison, it is likely that the CPI would have decreased in the pipelined program.

Additionally, the number of instructions executed decreased from over 80 to 51, which is a notable decrease in time.

Benchmark multiply_time

Benchmark Characteristics

Instruction Mix

44% Mem, 22% ALU, 22% Conditional, 11% Other

Number Instructions

9

Performance

Number Cycles

83

Instruction mix

49% Mem, 24% ALU, 25% Conditional, 1% Other

Comparison

Again, the CPI is technically greater, but if the previous ISA did not assume a default CPI of 1, it is likely that the CPI would have decreased with this pipelined program. One thing that was surprising in this benchmark is that the number of instructions executed stayed the same in the single cycle

-Discussion –

When working with the pipeline, we found the results to be mostly in line with our expectations. However, we were surprised by the complexity involved in setting up pipelining, considering how straightforward the concept is. Overall, the results matched our predictions. For our first project, ISA Simple 12, we assumed an ideal CPI (Cycles Per Instruction) of one. However, it was unlikely that we would achieve this with pipelining. Despite this, the CPI remained relatively low, especially considering that we dropped the ideal assumption and faced

hazards that interfered with the overall CPI of the program. We gained a much clearer understanding of pipelining. While the book provided a broad overview, we were able to observe the effects of pipelining and the challenges involved in its implementation. We had to balance stacking while also working to avoid structural hazards and data hazards. This experience allowed us to see just how effective pipelining can be in the field of computer architecture.

Following the implementation, we found the results show how effectively the utility of pipelining can reduce both processing time and unnecessary instructions. In our original program, the CPI was set at 1, while the pipelined program had a CPI ranging from 1.6 to 1.8. Although this indicates an increase in CPI, it can be attributed to stalls and branch hazards that temporarily slow down the program's performance. From this limited scope, we can speculate on the effectiveness of pipelining in larger computer architectures. Since pipelines are capable of executing instructions simultaneously, they can significantly reduce the time required to run instructions.

Given the nature of pipelining, it is expected that non-accumulator-based architectures would be more effective when used with a, such as the one we've been using. MIPS, being a register-based architecture designed with pipelining in mind, showcases this effectiveness. One of the key advantages is that all instructions are of the same length, which facilitates the fetching of instructions in the first stage and decoding them in the second stage without disrupting the pipeline's timeline.

Another design feature of MIPS that enhances pipelining is the consistent location of source registers across all instructions. This symmetry allows the program to read the next line of code while still fetching the current instruction, which aligns perfectly with how pipelining operates. Consequently, no extra steps are required for the pipeline to function as expected.

Additionally, as we learned previously, pipelining works best when operands in memory are aligned, since some architectures necessitate accessing memory twice. However, in MIPS, memory is aligned, so only a single stage is needed to transfer information to and from memory and the processor.

-Conclusion and Future Work

This project involved many learning curves. Before fully implementing the program, we established benchmarks for `'multiply_space'` and `'multiply_time'`. During the simulation of the control path, we discovered that a significant portion of the simulation was more challenging than we had anticipated, especially since we are still relatively new to the concept of pipelining.

We found it quite difficult to avoid all the hazards and stalls that occurred. Instead of trying to eliminate them entirely, we focused on mitigating their impact as much as possible. Moving forward, if we were to undertake a similar project, it might be more beneficial to concentrate on avoiding hazards and stalls from the outset rather than just working to overcome them. This approach would certainly require us to gain a deeper understanding of how pipelines function internally.

Additionally, if we pursue another project like this, it would be interesting to explore the effects of changing the number of stages in the pipeline. For our current project, we utilized a five-stage pipeline for the sake of time and our current level of understanding. However, experimenting with a six-stage or eight-stage pipeline could yield intriguing results, particularly in terms of its impact on each piece of data and how it affects the overall complexity of the project.

Work Cited

Patterson, David A., and John L. Hennessy. *COMPUTER ORGANIZATION and DESIGN MIPS EDITION: The Hardware/Software Interface*. 6th ed., S.L., Morgan Kaufmann Publisher, 24 Nov. 2020, pp. 1–2360.